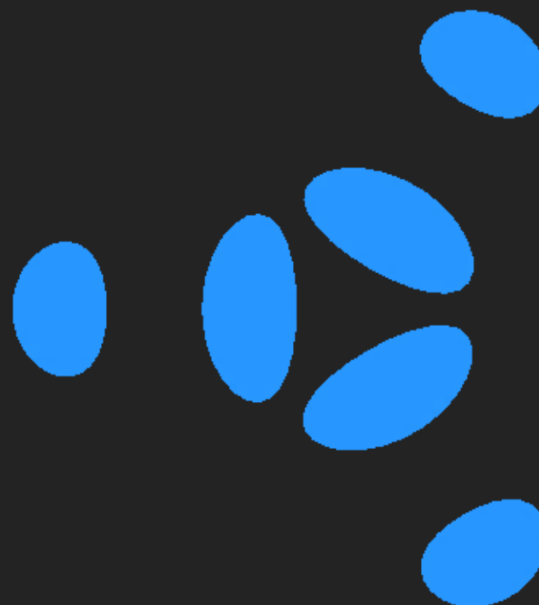




***COMEÇAREMOS
EM BREVE!***





especialização

RAN Intelligent Controller (RIC)

Prof. Dr. Jonas Augusto Kunzler

Open RAN:

Redes Abertas e Tecnologias Emergentes

 **Bloco 1 — 18:30–20:00**

OpenAirInterface — pilha, histórico e deploy

Disciplina 7: RAN Intelligent Controller (RIC)

Aula 04: Pilha OAI + interface E2 (FlexRIC)

Data: 16/06/2026 | Horário: 18:30–22:00 | Duração: 3h30

OpenAirInterface (core + RAN) · **deploy** e interface E2 via FlexRIC
NGO §3 (contexto OSS) · início **Projeto 2**





Calendário — destaque Aula 04

Data / Horário	Conteúdos
09/06 18:30–22:00	Aula 01: Retomada O-RAN + Lab Open5GS/UE-RAN-SIM
11/06 18:30–22:00	Aula 02: Artigo NGO + intro srsRAN/ZMQ
13/06 08:00–13:00	Aula 03: Seminários Polese+NGO + A1 + lab ZMQ
16/06 18:30–22:00	Aula 04: OAI+FlexRIC — início Projeto 2
18/06 18:30–22:00	Aula 05: Design de xApps e SMO
20/06 08:00–13:00	Aula 06: Testes, depuração e seminários

Acordos Didáticos

- Presença: início, **dois intervalos de 10 min** e encerramento.
- Registro de observações sobre deploy open source para o projeto em grupo.

Cronograma Previsto — Aula 04

- 18:30–20:00 — OAI: histórico, pilha, características · deploy (`oai-cn-gnb`)
- 20:00–20:10 — Intervalo
- 20:10–20:50 — Ciclo de vida · NGO §3.2–3.4 open source
- 21:00–21:10 — Intervalo
- 21:10–22:00 — Demo lab E2 · NGO §3 OSS · Projeto 2 · síntese

Objetivos da Aula 04

Objetivo central

Contextualizar a pilha OAI e operar a interface E2 via FlexRIC no laboratório

Ao final da aula

Subir Core+RIC+gNB; executar testes KPM/RC; iniciar Projeto 2 com roteiro documentado

Fio condutor — da simulação ao E2 real (OAI)

- **Aulas 01–03:** Open5GS → srsRAN ZMQ → desagregação CU/DU (sem E2 exposto).
- **Aula 04+:** pilha **OAI** com **agente E2** integrado ao gNB.
- **FlexRIC** como Near-RT RIC — subscriptions KPM e RC.
- Projeto: `code/oai-cn-gnb-e2/` · doc: `docs/E2_FLEXRIC.md`

OpenAirInterface — histórico e ecossistema

- **Origem:** Eurecom / OAI Alliance — stack 4G/5G open source desde ~2000s.
- **Licença:** OAI Public License v1.1 — uso acadêmico e industrial com atribuição.
- **Comunidade:** operadores, vendors e universidades; integração O-RAN SC e TIP.
- **Papel na disciplina:** pilha do curso com **E2 agent** maduro + RFSIM.

Pilha OAI neste laboratório

Camada	Componente	Deploy
5G Core	AMF, SMF, NRF, UPF-VPP, UDM, UDR, AUSF	Docker (<code>oai-cn5g-fed</code>)
RAN	<code>nr-softmodem</code> + <code>nr-uesoftmodem</code>	Nativo (host)
RF	RFSIM — simulador de rádio em software	Sem SDR/USRP
E2	Agente no gNB + FlexRIC nearRT-RIC	Nativo (host)

Características relevantes para o RIC

- **gNB monolítico** com E2 — cenário didático; CU/DU split também suportado upstream.
- **RFSIM**: attach, PDU session e métricas sem hardware — ideal para xApps.
- **E2SM-KPM v2.03** e **E2SM-RC v1.03** (O-RAN) + SMs custom L2/L3.
- **Slice lab**: SST=222, SD=123 — alinhado Core, UE e filtro KPM.
- **Contraste NGO §2**: poucos RAN comerciais expõem E2; OAI+FlexRIC é referência acadêmica.

Projeto `oai-cn-gnb-e2` — estrutura

- `oai-cn5g-fed/` — 5GC em Docker (`up_core.sh`)
- `openairinterface5g/` — RAN + submodule FlexRIC em `E2AP/flexric/`
- `flexric-lib/` — Service Models (`.so`) geradas pelo build
- `scripts/` — build, deploy e testes E2 automatizados
- `logs/` — `gnb_oai.log` , `nearRT-RIC.log` , `xapp_*.log`

Deploy — build (uma vez)

1. Dependências OAI: `./build_oai --ninja -I` (em `cmake_targets/`)
2. `./scripts/build_e2.sh` — gNB/nrUE com `--build-e2` (E2AP_V2, KPM_V2_03)
3. `./scripts/build_flexric_tools.sh` — nearRT-RIC, xApps, SMs → `flexric-lib/`

Versões E2AP/E2SM devem ser idênticas em gNB, RIC e xApps.

Deploy — ordem de subida

1. `./scripts/up_core.sh` — Core OAI (interface `demo-oai`)
2. `./scripts/up_flexric.sh` — nearRT-RIC em `127.0.0.1:36421`
3. `./scripts/up_gnb_oai.sh` — gNB + nrUE (E2 agent → RIC)
4. xApps: `test_e2_sm.sh` , `test_e2_kpm.sh` , `test_e2_rc_attach.sh`

Atalho: `./scripts/up_e2_lab.sh` · **Parar:** `down_e2_lab.sh`

Interface E2 — onde entra na arquitetura O-RAN

- **E2 node:** gNB OAI expõe RAN Functions (KPM, RC, MAC, RLC...).
- **Near-RT RIC:** FlexRIC roteia subscriptions e INDICATIONs.
- **xApp:** consome métricas/eventos; pode enviar CONTROL (RC PoC).
- **Fluxo:** **SETUP** → **SUBSCRIPTION** → **INDICATION** (periódica ou evento).

 **Bloco 1 (cont.) — 18:30–20:00**

NGO §3 — panorama RIC OSS (contexto)

Fio condutor — NGO (2024) + Polese (2023)

- **NGO §3:** survey de implementações — comercial e open source.
- **Polese:** plataformas, testbeds e desafios de adoção O-RAN.
- **Objetivo:** escolher stack para lab/projeto com critérios explícitos.

NGO §3 — soluções comerciais (closed source)

Fornecedor	Papel no ecossistema	Nota didática
Juniper	RIC / automação RAN	Integração operador; menos detalhe no NGO
VMware	RIC em plataforma telco cloud	Contraste com OSS em K8s
Capgemini	Integração e serviços RIC	Gap E2 em RAN comercial (§2)
Mavenir	RIC / O-RAN stack	Validação campo vs lab acadêmico

Artigo cita nomes; foco da disciplina é reproduzir fluxos em OSS.

Lacuna do setor (NGO §2 + Polese)

- Poucos RAN comerciais expõem **E2** conforme O-RAN.
- Integração exige proxy E2, customização E2SM ou RANsim (SD-RAN).
- OSS permite validar arquitetura; comercial traz SLA e suporte.
- NGO validou em **rede privada 5G real** — nosso lab principal é **OAI+FlexRIC** (E2).

De arquitetura E2 → aplicações no RIC

Até aqui: pilha OAI + interface E2 + FlexRIC.

Agora: o que são as **aplicações** que usam E2 e como nascem, sobem e evoluem no RIC.

- **xApp** — roda no **Near-RT RIC**; fala **E2** com o gNB.
- **rApp** — roda no **Non-RT RIC**; fala **A1** (políticas) e consome dados do SMO.
- NGO §3 e Polese comparam *plataformas*; o ciclo de vida descreve *como uma app entra e permanece* nessas plataformas.

xApp e rApp — definições (O-RAN)

	xApp	rApp
Onde roda	Near-RT RIC (<1 s)	Non-RT RIC (>1 s)
Interface principal	E2 (E2AP + E2SM)	A1 + dados SMO (O1, R1)
Objetivo típico	Monitorar/controlar RAN em quase tempo real	Analytics, ML, políticas de longo prazo
Exemplo no curso	xapp_kpm_moni (FlexRIC)	rApps SUTD / OSC (Aula 03)
Lab desta aula	Sim — OAI + FlexRIC	Contexto; aprofundamos na Aula 05

Ciclo de vida — as cinco fases (visão geral)

1. **Design** — definir o *quê* a app faz: E2SM (KPM, RC...), métricas, SLAs, políticas A1.
2. **Develop** — implementar com SDK/API do RIC; testes unitários e em emulador.
3. **Onboard** — registrar na plataforma: metadados, permissões, dependências E2SM.
4. **Deploy / Activate** — instanciar e conectar ao RIC em lab ou produção.
5. **Monitor / Upgrade / Retire** — observar, versionar, desligar com segurança.

NGO §3.2–3.4 · O-RAN SC rApp/xApp catalog · Polese — adoption challenges.

Ciclo de vida — o que acontece em cada fase?

Fase	Perguntas a responder	Artefato típico
Design	Quais KPIs? Qual E2SM? Latência aceitável?	Doc de requisitos, escolha KPM vs RC
Develop	Como subscrever? Como tratar INDICATION?	Código xApp, testes, README
Onboard	Quais RAN functions precisa? Quem pode rodar?	Descriptor / manifest (OSC) ou binário versionado
Deploy	RIC no ar? gNB com E2? Versões E2AP alinhadas?	Scripts <code>up_e2_lab.sh</code> , logs E2 SETUP
Monitor	Métricas chegam? Drift? Rollback?	Logs, painéis, evidências do Projeto 2

E2SM — o “contrato” do xApp com o gNB

- **E2AP:** transporte (SETUP, SUBSCRIPTION, INDICATION, CONTROL).
- **E2SM:** *semântica* — o que cada mensagem significa.
- **E2SM-KPM:** medições 3GPP (throughput, PRB...) — `test_e2_kpm.sh` .
- **E2SM-RC:** eventos/controle RRC — `test_e2_rc_attach.sh` .
- **Custom (MAC/RLC/PDCP/GTP):** camadas L2/L3 — `test_e2_sm.sh cust` .

No **Design** você escolhe o E2SM; no **Develop** implementa a subscription; no **Deploy** valida INDICATIONs.

No lab OAI+FlexRIC — qual fase exercitamos hoje?

Fase	O que fazemos na Aula 04
Design	Escolhemos KPM (métricas) e RC (eventos RRC) para o Projeto 2
Develop	Usamos xApps de referência (<code>xapp_kpm_moni</code>); extensão = customizar
Onboard	Simplificado: binário + <code>flexric-lib/</code> (sem catálogo K8s)
Deploy	Foco da demo: Core → RIC → gNB → xApp
Monitor	Capturar logs: E2 SETUP, subscription, INDICATION

Produção vs. nosso laboratório

Aspecto	O-RAN SC (produção)	FlexRIC + OAI (nosso lab)
Empacotamento	Helm chart, container, descriptor YAML	Binário nativo + <code>.so</code> em <code>flexric-lib/</code>
Isolamento	Pod K8s, limites CPU/mem, RBAC	Processo no host; <code>sudo</code> no gNB
Catálogo	rApp/xApp marketplace no SMO	Pasta <code>build/examples/xApp/</code>
Certificação	Testes de conformidade O-RAN	Scripts <code>verify_e2_lab.sh</code>

Fluxo E2 no ciclo de vida do xApp (lab)

1. **Deploy RIC + gNB** — E2 SETUP (gNB anuncia RAN functions KPM, RC...).
2. **Start xApp** — E42 SETUP com nearRT-RIC (porta 36422 → iApp).
3. **Subscribe** — xApp pede INDICATIONs (ex.: KPM slice 222/123).
4. **Operate** — métricas/eventos fluem; ping gera throughput mensurável.
5. **Retire** — SUBSCRIPTION DELETE; `down_e2_lab.sh`.

Comandos: `./scripts/test_e2_kpm.sh` · doc: `code/oai-cn-gnb-e2/docs/E2_FLEXRIC.md`

Discussão guiada — Projeto 2 e ciclo de vida

1. Qual **E2SM** seu grupo acha mais relevante (KPM, RC ou custom) e por quê?
2. Que evidência de **Deploy + Monitor** pretendem focar no Projeto 2? (log E2 SETUP + INDICATION)
3. Que limitação do **RFSIM** declarar na fase de Design/Monitor?
4. Se amanhã migrasse para OSC em K8s, o que mudaria só na fase **Onboard/Deploy**?



Intervalo — 20:00–20:10

 Bloco 2 — 20:10–20:50

Ciclo de vida · NGO §3.2–3.4 open source

Ciclo de vida — xApps e rApps

Já vimos as **5 fases** e o mapeamento ao lab OAI+FlexRIC.

Neste bloco: **onde** cada plataforma OSS encaixa no **Deploy/Onboard** — OSC, FlexRIC, SD-RAN.

- **FlexRIC** — Deploy direto (binário); nosso lab E2.
- **OSC** — Onboard via catálogo K8s; Non-RT + Near-RT.
- **Monitor** — DMaaP, InfluxDB (OSC); logs + `verify_e2_lab.sh`.

NGO §3 — comparativo open source (Near-RT + Non-RT)

Plataforma	Escopo	Prós (NGO)	Contras (NGO)
OSC	Non-RT + Near-RT	O-RAN compliant, escala K8s	Complexidade, muitos pods
FlexRIC	Near-RT	Lean, proxy E2, baixa latência	Monolito, escala limitada
SD-RAN	Near-RT + RANsim	SDK Go/Python, lab rápido	Sem PHY/user plane real

- nearRT-RIC acadêmico/industrial; E2SM KPM e RC.
- xApps em C: `xapp_kpm_moni` , `xapp_kpm_rc` .
- **Ordem de subida:** Core → RIC → gNB → xApp.
- Referência: NGO et al., ICT Express.
- Roteiro lab: `code/oai-cn-gnb/docs/E2_FLEXRIC.md`
- E2 agent OAI: `code/oai-cn-gnb/openairinterface5g/openair2/E2AP/`
`README.md`

OAI CN + GNB + FlexRIC — laboratório da disciplina

- **Core:** OAI 5GC (`oai-cn-gnb-e2/oai-cn5g-fed`) — `./scripts/up_core.sh`
- **RAN:** OAI gNB + nrUE com **RFSIM** — `./scripts/up_gnb_oai.sh`
- **Near-RT RIC:** FlexRIC (mesmas versões E2AP/E2SM-KPM do build OAI)
- **xApps:** `xapp_kpm_moni` , `xapp_kpm_rc` (conforme build)
- **Roteiro:** `docs/labs/02-oai-cn-gnb-flexric-e2.md`

Versões e alinhamento

Componente	Versão / branch
E2AP	v2 (`E2AP_V2`)
E2SM-KPM	v2.03 (`KPM_V2_03`)
FlexRIC	Submodule branch `dev`
gNB	`nr-softmodem` compilado com `--build-e2`
Modo RAN	RFSIM (sem hardware RF)
Core	`start-basic-vpp` cenário 1

gNB, nearRT-RIC e xApps devem partilhar a mesma stack E2AP.

Configuração do slice lab

Parâmetro	Valor
PLMN	208 / 95
S-NSSAI	SST 222 · SD 123
IMSI UE	`208950000000032`
AMF (host)	`192.168.70.129` · iface `demo-oai`
nearRT-RIC	`127.0.0.1:36421`

Ficheiros: `openairinterface5g/scripts/gnb.conf` · `ue.conf`

Resultados — visão geral

Procedimento	Estado
Core OAI + attach UE	✓
Build gNB E2 + FlexRIC tools	✓
E2 SETUP gNB ↔ RIC	✓
Custom SMs (142–148)	✓
E2SM-RC — INDICATION RRC	✓
E2SM-KPM — métricas periódicas	✓
PoC KPM+RC (`xapp_kpm_rc`)	⚠
SLICE / TC emulators	✗ N/A

E2 SETUP — ligação estabelecida

```
[E2 AGENT]: nearRT-RIC IP = 127.0.0.1, PORT = 36421  
           RAN type = ngran_gNB, nb_id = 3584  
[E2 AGENT]: Opening plugin .../flexric-lib/libkpm_sm.so  
[E2 AGENT]: Opening plugin .../flexric-lib/librc_sm.so  
[E2-AGENT]: E2 SETUP-REQUEST tx  
[E2-AGENT]: E2 SETUP RESPONSE rx
```

RAN Functions registadas: KPM · RC · SMs L2/L3

Custom Service Models

xApp: `xapp\_cust\_moni` · encoding plain · independente do slice

```
Connected E2 nodes = 1
Registered node 0 ran func id = 2    # KPM
Registered node 0 ran func id = 3    # RC
Registered node 0 ran func id = 142  # MAC
Registered node 0 ran func id = 143  # RLC
Registered node 0 ran func id = 144  # PDCP
Registered node 0 ran func id = 148  # GTP
```

✓ Confirma stack E2 end-to-end antes dos testes O-RAN.

E2SM-RC — subscrição e evento RRC

Script: `./scripts/test_e2_rc_attach.sh``

Ordem: RIC → xApp RC → gNB → UE(subscrição antes do attach)

[xApp]: Successfully subscribed to RAN_FUNC_ID 3

```
1 RC Indication Message received:
RAN Parameter Name = RRC Message
  <rrcSetupComplete>
    <rrc-TransactionIdentifier>1</rrc-TransactionIdentifier>
  </rrcSetupComplete>
```

✓ Mensagem RRC decodificada em ASN.1 · evento `RRCSetupComplete`

E2SM-RC — nota operacional

O `xapp_rc_moni` pode terminar com **timeout** em `sync_ui.c` após a 1.^a INDICATION — comportamento conhecido do exemplo FlexRIC.

Aspecto	Detalhe
Impacto	INDICATION já capturada antes do exit
Mitigação	Usar <code>test_e2_rc_attach.sh</code> · inspecionar log
Log	<code>logs/xapp_rc_attach.log</code>

E2SM-KPM — subscrição slice 222/123

Script: `./scripts/test_e2_kpm.sh`

Filtro: `KPM_SST=222` · `KPM_SD=123` · Report Style 4 (S-NSSAI)

✓ INDICATIONs periódicas (~1 s) com métricas 3GPP

E2SM-KPM — métricas observadas

Métrica	Exemplo	Unidade
DRB.UEThpDL	3.72 – 18.04	kbps
DRB.UEThpUL	3.72 – 19.18	kbps
RRU.PrbTotUL	2	%
DRB.RlcSduDelayDL	9 – 141	µs
DRB.PdcpSduVolume*	0*	Mb

Volume PDCP zero em RFSIM leve; `KPM\_TRAFFIC=1` (ping ao DN) aumenta throughput.



Intervalo — 20:50–21:00

 **Bloco 3 — 21:00–22:00**

Near-RT RIC, demo e projeto

Near-RT RIC (OAI + FlexRIC)

- **E2SM** — `code/oai-cn-gnb/docs/E2_SERVICE_MODELS.md`
- **E2 agent** — `code/oai-cn-gnb/openairinterface5g/openair2/E2AP/README.md`
- **xApps** — `xapp_kpm_moni`, `xapp_kpm_rc`
- **Roteiro** — `docs/labs/02-oai-cn-gnb-flexric-e2.md`

Demo guiada — laboratório E2

1. `cd code/oai-cn-gnb && ./scripts/up_e2_lab.sh`
2. `./scripts/test_e2_sm.sh cust` — validar RAN functions (MAC...KPM, RC)
3. `./scripts/test_e2_kpm.sh` — INDICATIONs KPM (slice 222/123)
4. `./scripts/test_e2_rc_attach.sh` — eventos RRC via E2SM-RC

Roteiro demo: `docs/labs/03-demo-e2-aula04.md` · guia
técnico: `code/oai-cn-gnb/docs/E2_FLEXRIC.md`

Projeto 2 (40%) — roteiro e prazos

- Implementar `oai-cn-gnb` conforme `docs/E2_FLEXRIC.md`
- Relatório técnico + demo (vídeo/logs) · extensão: xApp customizado ou caso A1/políticas.
- Apresentação na **Aula 06** (08:00–11:00), **mesma ordem** do Projeto 1, 20 min/grupo.
- Rubrica: `docs/avaliacao_seminario_aula06.md` · roteiro: `docs/labs/02-oai-cn-gnb-flexric-e2.md` · plano de testes (Aula 05): `docs/labs/04-projeto2-plano-testes.md`

Material de apoio — Aula 04

- NGO et al., ICT Express 2024 — §3 completo.
- Polese et al. — platforms, testbeds, open-source survey.
- Roteiros — `docs/labs/02-oai-cn-gnb-flexric-e2.md` , `03-demo-e2-aula04.md` ,
`04-projeto2-plano-testes.md`
- OAI+FlexRIC — `code/oai-cn-gnb/docs/E2_FLEXRIC.md`
- E2SM — `code/oai-cn-gnb/docs/E2_SERVICE_MODELS.md`

Próxima aula

Aula 05 — NGO Fig. 6: design integrado + SMO (Polese)

- Arquitetura SUTD: SMO ↔ Non-RT ↔ Near-RT ↔ RAN
- Workshop Projeto 2 — `docs/labs/04-projeto2-plano-testes.md` (NGO §6)



Referências

NGO, M. V. et al. "RAN Intelligent Controller (RIC)..." *ICT Express* 10 (2024).
(§3 — plataformas comercial e OSS)

Polese, M. et al. "Understanding O-RAN." *IEEE ComST*, 2023.

CHOPRA, A. et al. *Open RAN: The Definitive Guide*. Wiley, 2023.



Referências complementares

O-RAN SC — o-ran-sc.org

OAI E2 lab — `code/oai-cn-gnb/docs/E2_FLEXRIC.md`

Roteiro disciplina — `docs/labs/02-oai-cn-gnb-flexric-e2.md`

Demo Aula 04 — `docs/labs/03-demo-e2-aula04.md`

Plano de testes — `docs/labs/04-projeto2-plano-testes.md`

OAI E2 tutorial — `code/oai-cn-gnb/openairinterface5g/openair2/E2AP/README.md`



Jonas Augusto Kunzler



Email: jak@cesar.school



LinkedIn: [jakunzler](#)



GitHub: [jakunzler](#)



Instagram: [@jakunzler](#)